СУДАЛГАА, ХЭРЭГЖИЛТ, ТУРШИЛТЫН ТАЙЛАН

Project 2 — The Sleeping Teaching Assistant
F.CSM302 Үйлдлийн систем | Бие даалт 1
Хуваарилалтын код: 10
Багийн гишүүд: Б. Манлайбаатар, Ж. Мэнд-Амгалан, Г. Оргилсайхан, Б. Түмэнбаяр
Багш: Г. Ганбат

Хураангуй

Нэг TA, олон оюутан, хязгаарлагдмал хүлээлгийн сандал бүхий синхрончлолын асуудлыг C/POSIX thread, mutex, semaphore болон FIFO дарааллаар шийдвэрлэсэн. Python Tkinter интерфейс ctypes-ээр C хөдөлгүүртэй холбогдож, 0.1 секундийн загварын цаг, бодит төлөв, event log, CSV экспорт харуулна. Linux дээр GCC-ээр shared library хөрвүүлж, нийлүүлсэн unittest багцын 8 тестийг ажиллуулахад бүгд PASS гарсан. Туршилтууд FIFO, сандлын хязгаар, TA sleep, seed/reset, үйлчилгээний хугацаа болон хүчингүй оролтыг хамарна.

Түлхүүр үг: Pthreads, semaphore, mutex, FIFO, synchronization, simulation, Tkinter.

1. Удиртгал

Олон thread нэг нөөцийг хуваалцах үед төлөвийн уялдаа, харилцан үл хамаарах үйлдлийн дараалал, хүлээлт болон сэрээлтийг зөв зохион байгуулах шаардлагатай. Энэхүү ажилд TA нэг удаад нэг оюутанд тусалдаг, бусад нь хязгаартай сандалд хүлээдэг загварыг боловсруулсан. Зорилго нь thread synchronization-ийн механизмыг бодит ажилладаг хөдөлгүүр, удирдах UI, давтан шалгах тестээр нотлох юм. CPU scheduler-ийн throughput/latency-г бодит системийн хэмжилт гэж тайлбарлахгүй.

2. Сурах бичгийн шаардлага

Нэг TA thread, n student thread; оюутан программчлах ба тусламж хүсэх үйлдлүүдийг давтана. TA сул үед work semaphore-аар сэрнэ; завгүй бол

сандалтай тохиолдолд FIFO queue-д орно, сандалгүй бол буцна. TA үйлчилгээ дуусахад queue-ийн эхний оюутанд шилжинэ; queue хоосон бол унтана. Анхны бодлогын гурван сандлыг UI дээр 0–20 болгон өөрчилж турших боломжтой. sleep()-ээр санамсаргүй ханын хугацаа хүлээхийг загварын tick-ээр орлуулсан нь UI-г pause/step/reset горимоор удирдах зорилготой загварчлалын сонголт.

3. Онолын үндэс

3.1 Thread, mutex, semaphore pthread_create() нь TA болон student thread-үүдийг үүсгэнэ. pthread_mutex_t mu нь current, q, qn, state, due, visits, тоолуур болон загварын цагийг нэгэн зэрэг өөрчлөхөөс хамгаална. work нь TA-г ирэлт хүртэл блоклодог; started нь TA тусламж эхэлснийг шууд ирсэн оюутанд баталгаажуулдаг; done[i] нь үйлчилгээ дууссаныг тухайн оюутанд мэдээлдэг. ta_tick, ticks[i], ack нь нэг алхмын ажлыг дуусгах handshake үүсгэнэ.

3.2 FIFO ба invariant Хүлээлгийн q массивын эхний элемент q[0] дараагийн үйлчлүүлэх оюутан. Шинэ оюутан q[qn++] байрлалд орж, үйлчилгээ эхлэхэд memmove-ээр queue урагшилна. Нийт багтаамж MAX=40; сандлын хязгаар 0–20. Гол invariant: 0 ≤ qn ≤ chairs; current нь queue-д байхгүй; state=2 оюутан 0 эсвэл 1; state=1 тоо qn-тэй тэнцүү; arrivals = rejected + completed + qn + (current ≥ 0 ? 1 : 0).

3.3 Deadlock болон fairness Хүлээлгийн semaphore-г mutex барьсан үед хүлээхгүй байхаар хэрэгжүүлсэн. UI controller tick өгөх бүрдээ ack авч, worker үйлдэл дууссаны дараа дараагийн tick-ийг эхлүүлнэ. Queue-д орсон оюутнууд FIFO-оор үйлчлүүлэх боловч сандалгүй буцсан оюутанд тодорхой хугацаанд заавал үйлчлэх баталгаа байхгүй. Кодын бүтэц нь чөлөөт зэрэгцээ scheduling-ийн бүх боломжит interleaving-ийг шалгах зориулалтгүй.

4. Системийн зохиомж

Өгөгдлийн урсгал: Tkinter app.py → Model (ctypes) → libta.so C API → mutex/semaphore-аар синхрончлогдсон TA болон student thread-үүд. Буцах урсгал: sim_snapshot() JSON → json.loads() → Canvas болон статистик; sim_events() CSV мөрүүд → UI log / export. UI нь queue эсвэл үйлчилгээний төлөвийг өөрөө тооцоолж зохиохгүй, хөдөлгүүрийн snapshot-ийг уншдаг.

Компонент	Үүрэг
src/engine.c	Thread, semaphore, mutex, FIFO, clock, event log, JSON snapshot
src/model.py	ctypes CDLL, API аргумент/буцах төрөл, JSON/CSV wrapper
src/app.py	Tkinter input validation, Start/Pause/Step/Reset, Canvas, export
tests/test_engine.py	unittest invariant болон boundary тестүүд

4.1 Төлөвийн машин

Student: Код (state=0) → Хүлээнэ (1) эсвэл Тусламж (2) → Дууссан (3) → Код (0). Сандалгүй бол state=0 хэвээр үлдэж дараагийн due хугацааг шинэчилнэ. TA: sleep (current=-1) → help_start → help_end → дараагийн queue оюутан эсвэл sleep

5. Хэрэгжилт

Функц	Бодит үүрэг
sim_init	Оролт шалгах, RNG seed, semaphore/thread үүсгэх, төлөв reset
student	tick хүлээх; due болсон үед шууд TA-д орох, queue-д орох эсвэл буцах
teacher	work хүлээх; ta_tick бүрт remaining бууруулах; дуусахад done болон FIFO шилжилт
sim_step	now++ ; эхлээд TA үйлчилгээ, дараа нь student tick-үүдийг ээлжлэн ack-тай гүйцэтгэх

sim_snapshot	mutex дор JSON төлөв үүсгэх
sim_events	CSV event log-ийг буцаах
sim_close	stop тавих, блоклогдсон thread-үүдийг сэрээх, join, semaphore destroy

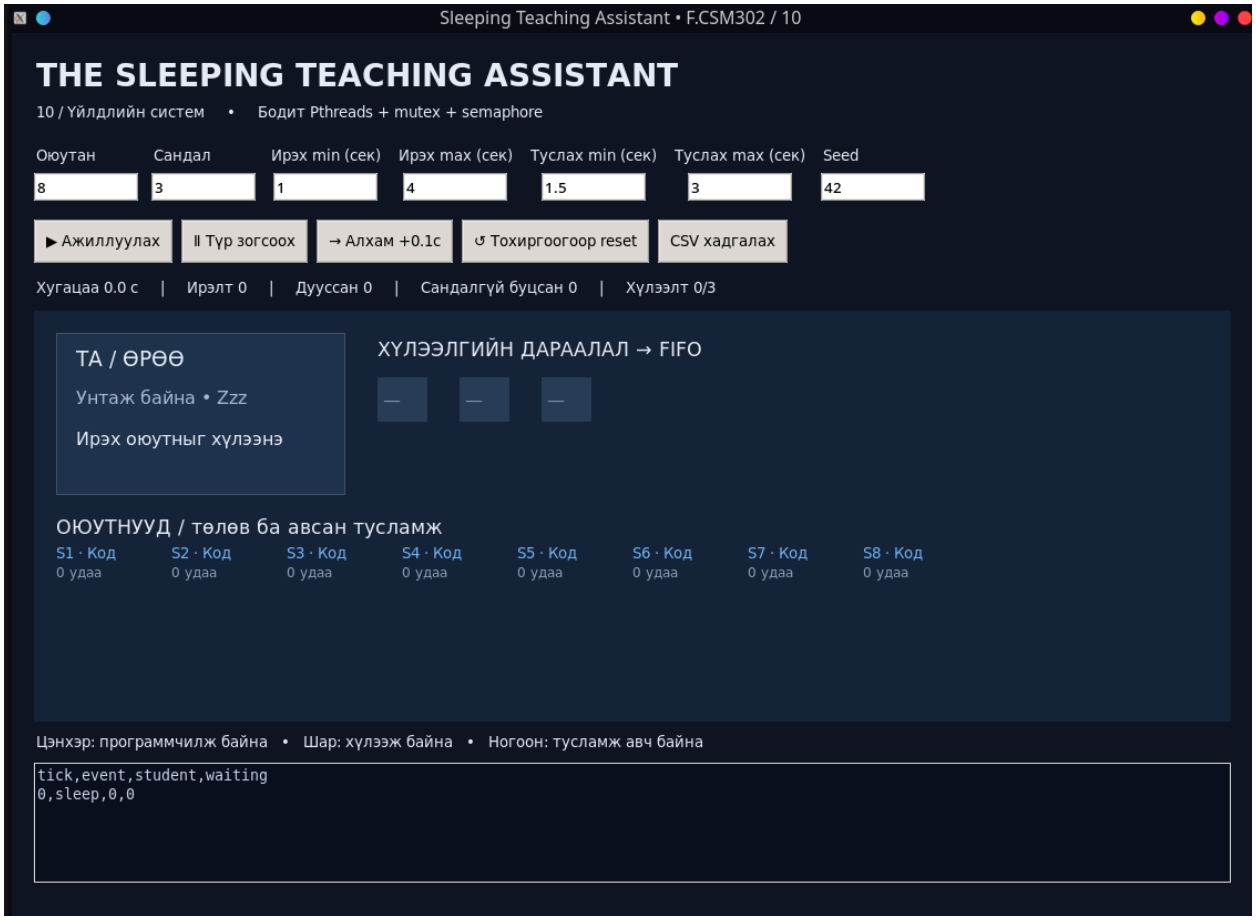
Санамсаргүй утгыг uint32_t LCG ашиглан үүсгэнэ. Загварын нэг tick = 0.1 секунд. Дотоод хугацааны утга tick нэгжтэй; UI секундийг 10-аар үржүүлэн tick болгодог. seed болон step-ийн тогтмол дараалал нь event log-ийг давтагдахуйц болгоно.

6. UI дүрслэл

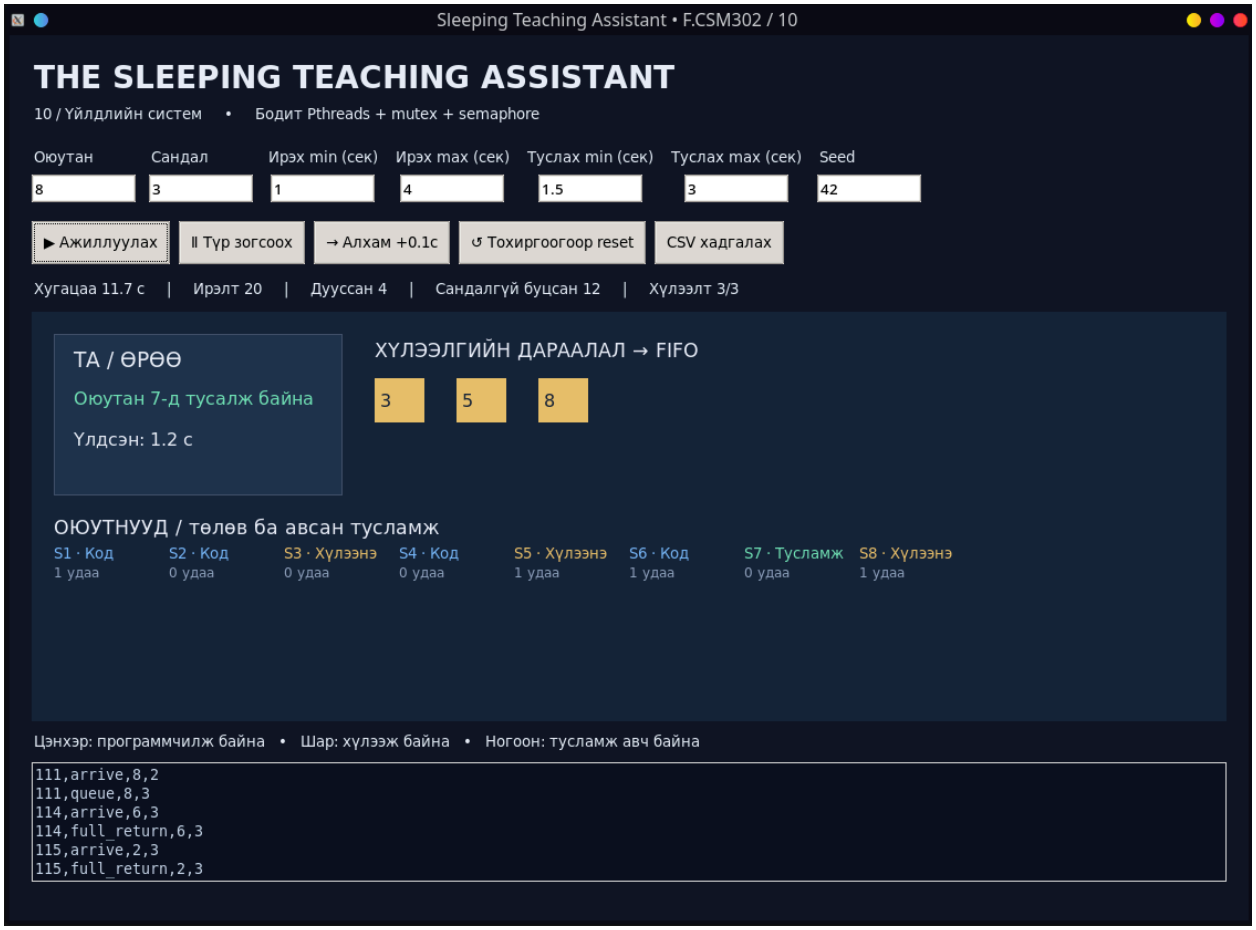
UI-д n=1–40, chairs=0–20, arrival/help 0.1–600 секунд (0.1 алхам), seed=0–4294967295 тохируулж болно. ▶ Run, Pause, Step +0.1c, Reset, CSV хадгалах товчтой. TA өрөө current/remaining, queue FIFO сандлууд, оюутан бүрийн төлөв, тусламжийн тоо, нийт ирэлт/буцалт/дуусалт харагдана. Өнгө: код цэнхэр, queue шар, тусламж ногоон, дууссан нил ягаан.

7. Туршилтын арга

Linux дээр gcc -std=c11 -O2 -Wall -Wextra -fPIC -shared -pthread src/engine.c -o libta.so командаар хөрвүүлсэн. Дараа нь python3 -m unittest discover -s tests -v командаар 8 тестийг гүйцэтгэсэн. Тест бүр 100–500 tick-ийн төлөвийг шалгаж, зарим нь event log-ийн FIFO болон үйлчилгээний хугацааг давхар баталгаажуулна.



Зураг 1: UI Үндсэн цонх харагдах байдал.



Зураг 2: Программ ажиллаж буй байдал.

```
mendamgalan@debian:~/OS/IV-2-10-sleeping-ta/sleeping-ta$ make test
python3 -m unittest discover -s tests -v
test_baseline_fifo (test_engine.EngineTests.test_baseline_fifo) ... ok
test_invalid (test_engine.EngineTests.test_invalid) ... ok
test_overload_and_maximum (test_engine.EngineTests.test_overload_and_maximum) ... ok
test_reproducible_and_reset (test_engine.EngineTests.test_reproducible_and_reset) ... ok
test_service_duration (test_engine.EngineTests.test_service_duration) ... ok
test_single_student_no_rejection (test_engine.EngineTests.test_single_student_no_rejection) ... ok
test_sleep_until_arrival (test_engine.EngineTests.test_sleep_until_arrival) ... ok
test_zero_chairs (test_engine.EngineTests.test_zero_chairs) ... ok

----------------------------------------------------------------------
Ran 8 tests in 1.202s

OK
```

Зураг 3: Test амжилттай гаралт.

8. Үр дүн

Тест	Туршилтын гол нөхцөл	Үр дүн
test_baseline_fifo	8 student, 3 chairs, FIFO event order	PASS
test_zero_chairs	10 student, 0 chairs, rejected > 0	PASS
test_single_student_no_rejection	1 student, 0 chairs, rejected=0	PASS
test_overload_and_maximum	40 student, 20 chairs, rejected > 0	PASS
test_sleep_until_arrival	99 tick TA sleep, 100-д ирэлт	PASS
test_reproducible_and_reset	3 удаа ижил seed → ижил log	PASS
test_invalid	6 төрлийн invalid input → ValueError	PASS
test_service_duration	5 tick үйлчилгээний яг хугацаа	PASS

Баталгаажуулалтын гаралт: Ran 8 tests in 0.745s; OK.
Энэ нь тест ажиллуулсан тусгаарласан орчны нэг удаагийн үр дүн бөгөөд бүх боломжит OS interleaving, UI platform болон хязгааргүй хугацааны найдвартай байдлыг нотлохгүй.

9. Хэлэлцүүлэг

Сайн тал: төлөвийг mutex дор хамгаалсан; TA-д зөвхөн нэг үйлчилгээ; FIFO queue; determinism; zero-chair болон max-load туршилт; GUI болон C engine-ийн үүргийн зааг тодорхой. Хязгаарлалт: симуляцийн цаг нь бодит wall-clock хэмжүүр биш; worker tick-үүд зориудаар дараалалтай; event buffer 262144 байт; controller нэг thread-ээс API дуудна; нөөц дутагдвал abort;

tick/counter overflow-ийг тусгайлан шалгаагүй. Хэрэглэгчийн зааварт тайлбарласан 0.1 алхам, seed, reset шалгалтууд UI түвшинд кодоор хэрэгжсэн боловч тусдаа GUI automation тестээр баталгаажаагүй.

10. Дүгнэлт

The Sleeping Teaching Assistant-ийн гол синхрончлолын шаардлагуудыг бодит POSIX thread, mutex, semaphore болон FIFO queue-ээр хэрэгжүүлж, тохируулгатай Tkinter UI болон CSV журналтай холбосон. Нийлүүлсэн 8 автомат тестийг Linux/GCC орчинд дахин ажиллуулахад 8/8 PASS гарсан. Үр дүн нь сургалтын зориулалттай давтагдах, алхамчилсан simulation-ийн correctness invariant-уудыг дэмжиж байна. Цаашид GUI automation, урт хугацааны overflow/log тест, CI, repository release болон бодит screenshot баримтжуулалтыг нэмэх нь зүйтэй.

Ашигласан материал:
1. OS_BD1_Guide.pdf — F.CSM302 Бие даалт 1, албан ёсны удирдамж.
2. Operating System Concepts — Programming Project 2: The Sleeping Teaching Assistant
3. OpenAI ChatGPT — баримт бичиг боловсруулах болон кодын тайлбар, шалгалтад ашигласан AI тусламж